Problem A
Russian Dolls

Input File: A.IN
Output File: standard output
Program Source File: A.C, A.CPP, A.JAVA

Maybe you know the famous russian souvenir Russian Dolls. It looks like a set of nested wooden dolls. A doll with a smaller size is placed inside a bigger one. Let's consider all dolls taken apart. Each doll has an outer volume out_i which is the volume it occupies in space and an inner volume in_i - the volume of the empty space inside the doll. You may assume that you can put one doll inside another if the outer volume of the first doll is strictly less than the inner volume of the second one. If two or more dolls are inside another one they can't lie one near the other, they must be nested.

For each doll the cost of unit of empty space - $cost_i$ is known. You must pay exactly $cost_i$ for each unit of empty space which directly belongs to the i-th doll (but not to ones inside it). You may arrange the dolls the way you want as long as you are not contradicting the rules. The objective is to find an arrangement of nesting the dolls (not necessarily all of them) such that the overall cost you have to pay is minimized.

Input
First line contains an integer N ($1 \le N \le 1000$) which is the number of dolls you have. The i-th of the next N lines contains three integers out_i, in_i, $cost_i$ ($1 \le in_i < out_i \le 1000$, $1 \le cost_i \le 1000$), which are the outer volume, inner volume and the empty space cost of the i-th doll.

Output
Single integer P which is the minimum possible cost you should pay.

Sample input	Sample output
3 5 4 1 4 2 2 3 2 1	7

Problem B
Intercity

Input File: B.IN
Output File: standard output
Program Source File: B.C, B.CPP, B.JAVA

A couple of years ago the Ukrainian Railway System seemed to be very convenient. For any two cities there was a direct train (don't be confused with "directed"), which travelled between them. Anyone had to pay only B UAH (the local currency) to get from his current place to the desired destination.

But not long ago great changes happened in Ukraine. A lot of new trains have been launched. Each of the new trains replaced an old one and the cost of its journey has been established to A UAH. So there is a single direct train (new or old) which still runs between any pair of cities now. Each train runs in both directions and the cost of the journey doesn't depend on it.

There are N large cities in Ukraine and you live in the city number 1. You want to get to the city number N, and you look for the cheapest way of going there, regardless of the number of transfers.

Input
The first line contains four integers N, K, A and B ($2 \le N \le 500000$, $0 \le K \le 500000$, $1 \le A, B \le 500000$), which are the number of cities, the number of new trains, the new cost of the journey and the old cost of the journey. K lines follow. Each of them contains two integers u_i and v_i ($1 \le u_i, v_i \le N$) which means that a new train is launched between city u_i and city v_i. u_i and v_i don't coincide. Every pair of cities appears at most once.

Output
Single integer P which is the cost of the cheapest way to go from city 1 to city N.

Sample input	Sample output
5 4 1 4 **1 2** **2 3** **2 4** **3 5**	**3**



Southeastern European Regional Programming Contest
Bucharest, Romania
October 12, 2013

Problem C
Count

Input File: C.IN
Output File: standard output
Program Source File: C.C, C.CPP, C.JAVA

You have:

- A matrix of natural numbers, with the property that all rows and all columns are sorted in ascending order (i.e. `A[i,j]>=A[i-1,j]` and `A[i,j]>=A[i,j-1]` for all `i,j`)
- One or several pairs of numbers `(X,Y)` with the property that `Y>=X`.

For each `(X,Y)` pair, count how many numbers from the matrix are greater than or equal to `X` but smaller than or equal to `Y`.

The input file is a binary file containing 32-bit integer numbers. The input file consists of:

- One integer `N` representing the number of rows (no more than `10000`)
- One integer `M` representing the number of columns (no more than `10000`)
- `NxM` integers, representing the values from the matrix, row by row
- An unspecified number of integers, representing the `(X,Y)` pairs, one pair at a time. There will be at least one pair and at most `100` pairs in the file – and there will not be an incomplete pair at the end of the file.

For each pair you should write to standard output a value representing how many numbers in the matrix are greater than or equal to `X` but smaller than or equal to `Y`.

Sample input (here in text form, not binary, for obvious reasons)	Sample output
2 4 **1 5 10 10** **2 10 20 99** **10 99** **2 9** **100 1000** **10 10**	**5** **2** **0** **3**

Problem D
Chess

Input File: D.IN
Output File: standard output
Program Source File: D.C, D.CPP, D.JAVA

There are two white rooks (♖) and one black king (♚) on a chess board. `White` starts. You're playing for `White` and need to checkmate the black king as soon as possible.
Write a program that will determine the minimum number of moves `White` needs to perform to checkmate black king, assuming `Black` follows the best possible strategy for prolonging the game.

Some information about the chess rules:

1. The position described above is not legal in chess since there is no white king on the board. Apart from that the game is according to the chess rules.
2. The board size is `8×8` cells. Columns of the board are labeled by the letters `a` to `h`, and the rows by the digits `1` to `8`.
3. The players (`White` and `Black`) alternately move one piece of their own color at a time. In the context of this problem we're only counting moves made by `White`.
4. The rook moves horizontally or vertically, through any number of unoccupied squares. It cannot jump over or stay in the same cell as another piece of the same color, namely the other white rook.
5. A check is a threat to capture the king on the next move turn, i.e. a position in which one of the rooks is on the same line as the king.
6. A king can move one square in any direction (horizontally, vertically, or diagonally) unless the move would place the king in check. If this condition holds, it's not prohibited for king to take over a cell already occupied by one of the rooks. In this case the rook is removed from play altogether.
7. Checkmate is a position in which a king is threatened with capture (i.e. is in check) and there is no legal move to escape the threat.
8. Stalemate is a situation where the player whose turn it is to move is not in check but has no legal move. The rules of chess provide that when stalemate occurs, the game ends as a draw (which is not acceptable for `White`).

Input
The first line of the input contains the number of positions (which is at least `1`), and each of the following lines contain the description of a single position. All positions in the input are different.
A position is represented by the location of the three pieces on the board: the black king first and then the two white rooks. It's guaranteed that no two pieces share a cell and there is no check at the initial position.

Output
Output one line for each position in the input. The line should contain the minimum number of moves which `White` needs to make to guarantee the checkmate, starting from the corresponding position. In case it's not possible to reach the goal, output `0` for relevant position.

Sample input	Sample output
2 **c7 f1 g6** **h6 c3 g8**	**2** **1**

Problem E
A+B

Input File: E.IN
Output File: standard output
Program Source File: E.C, E.CPP, E.JAVA

There is a computer, which has two memory cells (let us denote these cells by the letters `a` and `b`). Each cell (variable) stores some integer at any time. The computer can execute only two instructions `a+=b` and `b+=a`. The first instruction increases the value of the variable `a` by the value stored in the variable `b`. The second one, respectively, increases the value of `b` by the value `a`. A program for this computer consists of a sequence (possible empty) of such instructions. The instructions are executed in the appropriate order. Your task is to determine whether the given value `S` can be obtained in some cell after executing some program.

Input
The input file contains three integers: the initial value of the variable `a`, the initial value of the variable `b` and the required value `S` ($0 \le a, b, S \le 10^{18}$).

Output
Output `YES` if the required value can be obtained as a result of some program execution, or `NO` otherwise.

Sample input	Sample output
1 2 3	YES
3 4 5	NO
3 4 17	YES

Problem F
Dragon Pattern

Input File: F.IN
Output File: standard output
Program Source File: F.C, F.CPP, F.JAVA

The dragon curve is a polygonal chain with segments of length 1, defined recursively. We choose some point on the plane and one of four directions, collinear to the coordinate axes. The left(right) dragon of order n is constructed in the following way:

- if n is equal to 0, we construct a segment of length 1 from the current point in the current direction, moving to endpoint;
- otherwise we construct the left dragon of order n−1 from the current point in the current direction, turn 90 degrees to the left(right) in the endpoint, and construct the right dragon of order n−1;

Let us construct the left dragon of order n, starting from the origin (point (0,0)) in the positive direction of the axis OX.

Given some pattern as a sequence of directions, in which sequential movements by unit length is performed, your task is to determine how many times does the given pattern occurs in the constructed dragon curve.

Input
Input file contains integer n ($0 \le n \le 10^9$) and string S consisting of symbols R, L, U and D, which defines the pattern. R denotes rightward movement (positive direction of the axis OX), L denotes leftward movement (negative direction of the axis OX), U denotes upward movement (positive direction of the axis OY), D denotes downward movement (negative direction of the axis OY). The length of the string S is in the range from 1 to 10^6.

Output
Output number of given patterns in the left dragon of the order n, constructed in the positive direction of an axis OX, modulo 10^9+7.

Sample input	Sample output
2 RU	**1**
3 L	**3**
4 LDL	**2**



Southeastern European Regional Programming Contest
Bucharest, Romania
October 12, 2013

Problem G
Points

Input File: G.IN
Output File: standard output
Program Source File: G.C, G.CPP, G.JAVA

John is a fan of amusement parks. He goes every weekend and plays different games. This weekend he found a challenging one - it is a target shooting game. The targets are placed along a straight line. For all target positions i (assume the target numbering goes from right to left), there are three possible points that John can win if he chooses that target: a_i, if there are no neighbor targets chosen, b_i if one neighbor, and c_i if two neighbors. Could you help John choose the targets in order to maximize the number of points he can win?

The program input is from a text file. It starts with the number n ($n < 1000000$) of targets. Follows the values of a_i, b_i, and c_i for each target i. The program prints the maximum number of points John can win.

The input data are correct and terminate with an end of file. The program prints the result to the standard output from the beginning of a line.

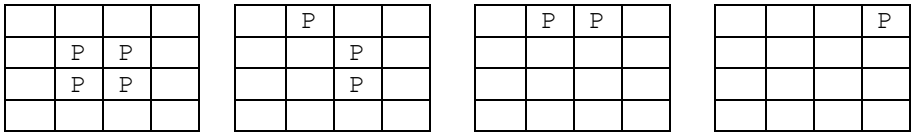
Input/output samples are in the table below. There are two tests. Each consists of only one target. For the first one $n=1$, $a_1=3$, $b_1=0$, $c_1=0$, and the maximum number of points is 3. For the second one $n=1$, $a_1=1$, $b_1=2$, $c_1=3$, and the maximum number of points is 1.The result consists of the maximum number of points John can win, printed from the beginning of the line.

Sample input	Sample output
1 3 0 0	3
1 1 2 3	1

Problem H
Red John Game

Input File: H.IN
Output File: standard output
Program Source File: H.C, H.CPP, H.JAVA

Red John has a chess table of infinite dimensions, and `n * n` pawns, arranged in an `n x n` square. The pawns can be moved horizontally or vertically, buy jumping over an (horizontally or vertically) adjacent pawn, and onto the next position, only if this position is unoccupied by another pawn. Also, when a valid move occurs, the jumped pawn is removed. Can you help Red John figure out if there is a sequence of moves which leaves only one pawn on the table ?
Below, such a sequence of moves is illustrated, for `n = 2`. Pawns are depicted by the letter `P`.



The program input is from a text file. Each file contains a value for `n`, with $0 < n < 10^9$. The output consists of `1` if there is a sequence of moves leaving only one pawn on the table, and `0` otherwise. There cannot be any whitespace and newline characters in the output. Two examples of input/output pairs are shown below.

Sample input	Sample output
3	**0**
4	**1**

Problem I
Influence

Input File: I.IN
Output File: standard output
Program Source File: I.C, I.CPP, I.JAVA

In any society there are social influence relations between individuals, where a person `x` can influence another person `y`. The same is true for Softopia, a special society where social influence relations are also transitive, meaning that if `x` can influence `y` and `y` can influence `z`, then `x` also influences `z`. Moreover, the rules of social influence guarantee that if `x` influences any other person `y`, then `x` cannot be influenced by `y` as well. Using these simple rules, if a person `x` from Softopia wants something, then all the nodes influenced by `x` also want the same thing.

Although, Softopia is almost a perfect society, there is a set of certain individuals, `X`, that would spread false demands through the social influence mechanism used by the society. And there is also an evil entity that would want to find out which of these people should be selected to spread a demand of their own. Because the society can only select one person from `X`, it would like to select one that is able to influence as many people as possible from Softopia. If there is more than a person that satisfies this request, you should pick the one with the lowest identifier.

Input data. The input file starts with a line containing two integers separated by one space: `n (n ≤ 5000)` the number of individuals in the society, and `k`, the number of elements in set `X`. The next line contains the elements from `X`, thus `k` different integers from `1..n` separated by one space. Then follow `n` lines and each line `i`, `1≤i≤n`, contains first the identifier of the current person followed by the identifiers of the persons that can be directly influenced by person `i`, all of them separated by a space. The persons are labeled from `1` to `n`. The total number of influences in a society is less than `250000`. Additional whitespaces in the input file should be skipped.

Output data. The result, representing the identifier of the person that satisfies the conditions mentioned above, will be written on a single line.

The table below describes two samples of input and output.

Sample input	Sample output
5 2 1 2 1 3 4 2 3 4 3 5 4 5 5	1
6 3 1 2 3 1 2 2 5 3 4 2 4 6 5 6	3

Problem J
An Idea of Mr. A

Input File: J.IN
Output File: standard output
Program Source File: J.C, J.CPP, J.JAVA

Mr. A proposes to his son the following problem:

"Consider two integers $\boldsymbol{n_1}$ and $\boldsymbol{n_2}$ such that $\boldsymbol{1 \le n_1 < n_2 \le 10^4}$. Using the function $\boldsymbol{p}: \mathbb{N}^* \to \mathbb{N}^*, \boldsymbol{p(n)} = \boldsymbol{2^n}, \forall \boldsymbol{n} \in \mathbb{N}^*$ (where $\mathbb{N}^*$ is the set of positive integers) we define the set

$$\boldsymbol{S(n_1, n_2)} = \{\boldsymbol{p(p(n))} + \boldsymbol{1} \mid \boldsymbol{n_1 \le n \le n_2}\}$$

We also define a set of pairs as follows:

$$\boldsymbol{T(n_1, n_2)} = \{(\boldsymbol{m_1, m_2}) \mid \boldsymbol{m_1, m_2} \in \boldsymbol{S(n_1, n_2)}, \boldsymbol{m_1 < m_2}\}$$

Consider the formula:

$$\boldsymbol{R(n_1, n_2)} = \sum_{(\boldsymbol{m_1, m_2}) \in \boldsymbol{T(n_1, n_2)}} \mathbf{gcd}(\boldsymbol{m_1, m_2})$$

where $\mathbf{gcd}(\boldsymbol{m_1, m_2})$ is the greatest common divisor of $\boldsymbol{m_1}$ and $\boldsymbol{m_2}$. The problem asks to find the number $\boldsymbol{R(n_1, n_2)}$."

Solve the problem proposed by Mr. A.

Input
The input file consists of a single line having the values for n_1 and n_2, separated by exactly one space.

Output
For each test, the result will be written to standard output and consists of the value of $\boldsymbol{R(n_1, n_2)}$.

Sample input	Sample output
1 34	561
15 147	8778
125 1000	383250